CS 348 Milestone 2

Crystal Zhang, Jessica Chen, Rosanne Zhu, Sabrina Xing, Siting Ban

PokePals Description

This project aims to create a database-driven web application for Pokemon card trading targeted to Pokemon enthusiasts. We plan to use the dataset "Pokemon TCG All Cards 1999 - 2023" and Pokemon Cards from Kaggle, which includes information about all of the Pokemon cards released over the years. This dataset provides attributes including the card names, rarity, release date, generation, and more.

On the application, users will be able to create/login to their own profile and search or filter through available cards by their attributes. If the user sees a Pokemon card that they really like, they can request to trade cards with the owner of that Pokemon card. Users will also be able to create their own cards to add to their collection as well. We, as the developers, are the administrators of the database and we will also select some diehard Pokemon fans as well.

Note:

- We will using a subset of the Pokemon TCG All Cards dataset as our production dataset it has 17,000+ entries

Planned Features

- Gift and request Pokemon cards to/from other users
- Obtain Pokemon cards through gambling
- Filter and search for Pokemon cards in the home page
- Each user has the ability to log in & sign up
- Each user has their own dashboard to view their information and the cards that they own

Note: highlighted features are the ones that are implemented

Optional Features

- Upload and create new and personalized Pokemon cards
- Trade Pokemon cards with other users

SQL Queries

Gift a Pokemon card

The user can gift their Pokemon cards to other users. They would lose ownership of that Pokemon card and the other user will receive a Pokemon card.

Query template:
INSERT INTO transaction (transaction_id, card_id, sender_id, receiver_id, date) VALUES ([card_id value], [sender_id value], [receiver_id value], [date value]);
DELETE FROM ownership WHERE uid = sender_id AND card_id = ownership.card_id
INSERT INTO ownership (uid, card_id) VALUES ([receiver_id value], [card_id value])

Sample query:
INSERT INTO transaction (card_id, sender_id, receiver_id, date) VALUES (22, 'bare-283', 'user-32', 'user-33', NOW());
DELETE FROM ownership WHERE uid = 'user-32' AND card_id = 'bare-283'
INSERT INTO ownership (uid, card_id) VALUES ('user-33', 'bare-283')

Expected Output:

transaction_id	card_id	sender_id	reciever_id	date
22	'bare-283'	'user-32'	'user-33'	2025-02-07 12:34:56

Create a new user
To begin using the site, users can create an account. They can login to the account to access their cards.

Query template:
INSERT INTO Account (uid, username, password, bio, pfp) VALUES ([uid], [username], [password], [bio], [pfp]);

Sample query:
INSERT INTO account (uid, username, password, bio, pfp)
VALUES (123456789, 'cool_username', 'password', 'New user', 'https://coolphoto.com/pfp');

Expected Output:

uid	username	password	bio	pfp
123456789	'cool_username'	'password'	'New user'	'https://coolphoto.com/pfp'

Create new Pokemon card
The users can make their own Pokemon cards which can then be traded.

Query template:
INSERT INTO `Pokemon Card` (card_id, pname, set_name, is_custom, image_url, generation, release_date, rarity, subtype, hp, level, flavour_text)
VALUES ([card_id], [pname], [set_name], TRUE, [image_url], [generation], [release_date], [rarity], [subtype], [hp], [lvl], [flavour_text]);

Sample query: *INSERT INTO pokemon_card (card_id, pname, set_name, is_custom, image_url, generation, release_date, rarity, subtype, hp, level, flavour_text) VALUES ('Poke123', 'Pikachu', 'Base Set', TRUE, 'https://example.com/pikachu.jpg', 'Generation I', 1999-01-09, 'Rare', 'Electric', 60, 12, 'A mouse Pokémon');*

Expected output:

card_id	pname	set_name	is_custom	image_rul	generation	release_date	rarity	subtype	hp	level	flavour_text
'Poke123'	*'Pikachu'*	*'Base Set'*	TRUE	*'https://example.com/pikachu.jpg'*	*'Generation I'*	*1999-01-09*	*'Rare'*	*'Electric'*	60	12	*'A mouse Pokémon'*

Search and filter for Pokemon cards
Users can search for Pokemons based on their attributes.

Query template: *SELECT * FROM pokemon_card WHERE [add queried attributes here];*

Sample template: *SELECT * FROM pokemon_card WHERE pname = 'Pikachu';*

Expected output:

card_id	pname	set_name	is_custom	image_rul	generation	release_date	rarity	subtype	hp	level	flavour_text
'Poke123'	*'Pikachu'*	*'Base Set'*	TRUE	*'https://example.com/pikachu.jpg'*	*'Generation I'*	*1999-01-09*	*'Rare'*	*'Electric'*	60	12	*'A mouse Pokémon'*

Gambling for Pokemon Cards
Users can add new Pokemon cards to their collection through gambling.
Sample Queries:
From our assumption that there is only one of each Pokemon Card in existence, we will filter for card_ids that are not currently owned

From the previous query we need to select a random card for the user. The user may draw 3 cards at once. Then, we will insert into the ownership table.
INSERT INTO ownership (uid, card_id)
SELECT account.uid, pokemon_card.card_id
FROM account, pokemon_card
WHERE account.uid = [userid] AND pokemon_card.card_id *NOT IN* (*SELECT* card_id from ownership)
ORDER BY RAND()
LIMIT 3;

Expected output:
3 additional entries to the Ownership Table
Ownership Table
… (previous entries)

uid	card_id
1	dp6-99
2	pl3-93
3	base2-37

For reference, the **card_id**s above are the pokemon below
('dp6-99', 'Hitmonchan', 'Legends Awakened', 0, 'https://images.pokemontcg.io/dp6/99_hires.png', 'Diamond & Pearl', datetime.date(2008, 8, 1), 'Common', "['Fighting']", "['Basic']", 70, 27, 'The arm-twisting punches it throws pulverize even concrete. It rests after three minutes of fighting.')

('pl3-93', 'Bulbasaur', 'Supreme Victors', 0, 'https://images.pokemontcg.io/pl3/93_hires.png', 'Platinum', datetime.date(2009, 8, 19), 'Common', "['Grass']", "['Basic']", 60, 13, 'For some time after its birth, it grows by gaining nourishment from the seed on its back.')

('base2-37', 'Gloom', 'Jungle', 0, 'https://images.pokemontcg.io/base2/37_hires.png', 'Base', datetime.date(1999, 6, 16), 'Uncommon', "['Grass']", "['Stage 1']", 60, 35, "The fluid that oozes from its mouth isn't drool; it is a nectar that is used to attract prey.")

Each user can view their own dashboard
After logging in, each user can view their profile and the cards that they own through their dashboard.

Query template:
*SELECT * FROM account WHERE uid = ‘user_id’*
*SELECT * FROM pokemon_card WHERE card_id IN (SELECT card_id FROM ownership WHERE uid = ‘user_id’)*

Sample query:
*SELECT * FROM account WHERE uid = ‘123456789’*
*SELECT * FROM pokemon_card WHERE card_id IN (SELECT card_id FROM ownership WHERE uid = ‘123456789’)*

Expected Output:

uid	username	password	bio	pfp
123456789	'cool_username'	'password'	'New user'	'https://coolphoto.com/pfp'

card_id	pname	set_name	is_custom	image_rul	generation	release_date	rarity	subtype	hp	level	flavour_text
'Poke123'	*'Pikachu'*	*'Base Set'*	TRUE	*'https://example.com/pikachu.jpg'*	*'Generation I'*	*1999-01-09*	*'Rare'*	*'Electric'*	60	12	*'A mouse Pokémon'*

Description of Platform

We will be hosting locally and users will interact through a TypeScript React web application front end and Python Flask backend which will connect to a MySQL database.

Sample Plan

Data will be populated through a python script into a MySQL database. We have cleaned up the data retrieved from the kaggle website and sorted them by their corresponding tables into CSV files. Through the python script, we can directly import the CSV file data into tables in MySQL.

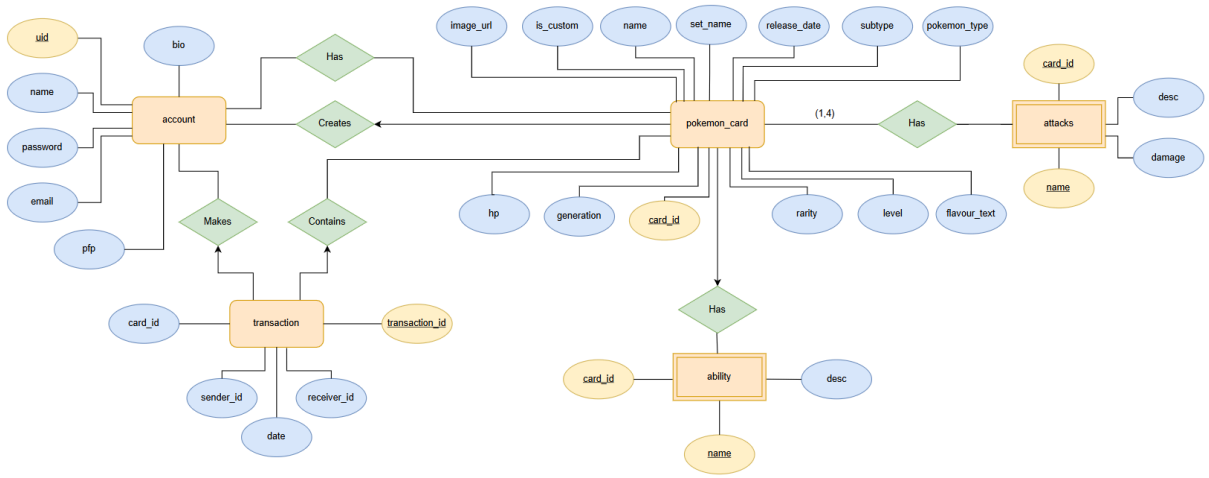
Assumptions

1) There is only one copy of every Pokemon card in existence
2) All cards are Pokemon. There are no energy, trainer, etc. cards.
3) Pokemon can belong to only one or two types
4) The pokemon id from Kaggle dataset (pokemon-tcg-data-master 1999-2023) is unique

5) The pokemon id from Kaggle dataset (pokemon-tcg-data-master 1999-2023) has a one-to-one match for the id from the Kaggle dataset (Pokemon Cards)
   Ex: a Pokemon with id: X from **pokemon-tcg-data-master 1999-2023** would be the exact same card as Pokemon with id: X from **pokemon cards**
6) Users are trading virtual Pokemon cards
7) Every Pokemon card only has one ability. But two or more Pokemon cards can have the same ability.
8) Pokemon Weakness and Resistances follows this chart:
   - https://www.thegamer.com/pokemon-types-weakness-chart/
9) The damage attribute in attack is the minimum damage a Pokemon can do. We will not consider actual damage because that is dependent on gameplay and non-Pokemon cards which is not in our assumption that all cards are Pokemon cards.

E/R Diagram

Please zoom in to view:
https://drive.google.com/file/d/1Fe7H5xObP88ulnN23j_kFt-Wdr700rpF/view?usp=sharing



- We decided to remove the weaknesses & resistances entities since it's redundant information that would be found on the the card, and people would not normally search for pokemon via their weaknesses or resistances

List of Tables (Relational Data Model)

Account

Attribute	Type	Constraints
uid	INT	Primary Key, auto increment

username	VARCHAR	
email	VARCHAR	
usr_password	VARCHAR	
bio	VARCHAR	
pfp	VARCHAR	

Ownership

Attribute	Type	Constraints
uid	INT	Foreign key to Account(uid)
card_id	VARCHAR	Foreign key to Pokemon Card(card_id)

Primary key: (uid, card_id)

Transaction

Attribute	Type	Constraints
tid	INT	Primary Key, auto increment
card_id	VARCHAR	Foreign key to Pokemon Card(card_id)
sender_id	DECIMAL (9,0)	Foreign key to Account(uid)
receiver_id	DECIMAL (9,0)	Foreign key to Account(uid)
tdate	DATETIME	

Pokemon Card

Attribute	Type	Constraints
card_id	VARCHAR	Primary Key

pname	VARCHAR	
set_name	VARCHAR	
is_custom	BOOLEAN	
image_url	VARCHAR	
generation	VARCHAR	
release_date	DATE	
rarity	VARCHAR	
pokemon_type	VARCHAR	
subtype	VARCHAR	
hp	INT	
level	INT	
flavour_text	VARCHAR	

Abilities

Attribute	Type	Constraints
card_id	VARCHAR	Foreign key to Pokemon Card(card_id)
ability_name	VARCHAR	
description	VARCHAR	

Primary key: (card_id, ability_name)

Attacks

Attribute	Type	Constraints
card_id	VARCHAR	Foreign key to Pokemon Card(card_id)
attack_name	VARCHAR	
description	VARCHAR	

damage	INT	

Primary key: (card_id, attack_name)

Members Contributions

Contributions of members for Milestone 2:

- Rosanne - contributed to report document, designed and implemented frontend (pages include login, signup, dashboard, gifting/requesting, adding cards), implemented login and signup authentication for account creation, connected searching/filtering from frontend and backend, completed testing for all features
- Sabrina - wrote backend API for requesting pokemon cards, contributed to report document and ideas for ER diagram and schema
- Siting - cleaned up CSV files obtained from Kaggle and sorted data into new CSV files by table, wrote backend API for searching/filtering pokemon cards, updated E/R diagram and report document
- Jessica - contributed to document, READMe, created python script for populating data into mysql db
- Crystal - contributed to report document and ideas for ER diagram and schema

GitHub Repo

https://github.com/sabrina-xing/pokemon